

面向对象程序设计

金融工程

课程主要内容

1. C++程序设计语言基本语法
2. 面向对象程序设计思想
3. 面向对象程序设计方法
 1. 封装性
 2. 继承性
 3. 多态性

0. 高级程序设计语言分类

- 按照程序设计语言执行方式：
 - 解释型语言：Python、Matlab等
 - 编译型语言：C++、C等
- 按照程序设计机制：
 - 面向**过程**的程序设计语言：C等
 - 面向过程的程序设计语言**不支持**面向对象的程序设计机制
 - 面向**对象**的程序设计语言：C++、Python、Java等
 - 面向对象的程序设计语言**支持**面向过程的程序设计机制

1. C++程序设计语言基础语法

1.1 C++语言程序创建过程

1. 编辑：源文件（.cpp文件）
2. 编译：高级语言 → 汇编语言 → 机器语言
3. 链接：关联相应头文件并生成可执行文件（.exe文件）
 - iostream
 - cmath
 - string
4. 执行

1.2 C++语言程序的基本结构

```
#include<iostream>
using namespace std;
int main()
{
    cout << "Hello world" << endl;
    return 0
}
```

- 头文件

iostream: input & output stream（输入输出流）

添加标准库头文件使用“#include<>”的方式

- 使用命名空间

使用“std”命名空间

cin、cout、endl标识符存放在“std”命名空间中

- 主函数

“int main() { return 0; }”是主函数的基本框架

“main”是函数的名字

“{}”是函数的内容

所有C++语言程序都必须包含一个主函数

- 指令

以分号结尾的都是一句指令

1.3 常用数据类型

- 整型 (int, 4字节)：对应**整数**

```
int a; // 创建整型变量a
int a = 1; // 创建整型变量a并赋值为1
int a = 1.99; // 创建整型变量a并赋值为1
```

- 浮点型：对应**小数**

- 单精度 (float, 4字节)
- 双精度 (double, 8字节)

```
float b; // 创建浮点型变量b
float b = 1; // 创建浮点型变量b并赋值为1.0
float b = 1.99; // 创建浮点型变量b并赋值为1.99
```

- 字符型 (char, 1字节)：对应**符号** (ASCII码表)

```
char c; // 创建字符型变量c
char c = '1'; // 创建字符型变量c并赋值为字符1
char c = 'a'; // 创建字符型变量c并赋值为字符a
char c = ' '; // 创建字符型变量c并赋值为字符空格
```

- 字符串型 (string)

- 需搭配头文件string使用

```
#include<string>
string s; // 创建字符串型变量s
string s = "hello world"; // 创建字符串型变量并赋值“hello world”
```

1.4 输入输出

- 输入

```
cin>>a>>b>>c;
```

- 输入流: cin
- 提取运算符: >>

- 输出

```
cout<<a<<b<<c<<endl;  
cout<<"hello world"<<endl;  
cout<<a<<" "<<b<<" "<<c<<endl;
```

- 输出流: cout
- 插入运算符: <<
- 换行: endl (end of the line)

1.5 常用算法结构

- 选择结构

```
if(){} else if(){} else{}
```

- 数据类型

- 布尔型 (bool, 1字节)
 - 仅有“真 (true) ”和“假 (false) ”
 - true → 1, 非0 → true
 - false ↔ 0

- 逻辑运算符

- 逻辑与 (&&)
- 逻辑或 (||)
- 逻辑非 (!)

```
int n;  
if(n>0 && n<10) // 等价于数学表达式0<n<10  
if(n==1 || n==3) // 表示n等于1或者3的情况
```

- 关系运算符 (>、>=、<、<=、==、!=)

- 等于==

```
if(n=1) // 变量n赋值为1, 为true  
if(n==1) // 判断变量n是否为1
```

```
// 选择结构示例：分段函数
int x, y;
cin>>x;
if(x<0)
{ y = -x; }
else if(0<=x && x<=10)
{ y = x-10; }
else
{ y = x; }
cout<<y<<endl;
```

- 循环结构

```
for(表达式1;表达式2;表达式3)
```

- 第1个表达式：设置循环变量起始值
- 第2个表达式：循环是否执行的条件
- 第3个表达式：更改循环变量

```
// 循环结构示例：输出1 2 3 4 5 6 7 8 9 10
int i;
for(i=1;i<=10;i++)
{
    cout<<i<<" ";
}
cout<<endl;
```

1.5 函数

- 函数的组成部分
 - 函数名
 - 函数参数（形式参数）
 - 函数类型/函数返回值类型：对应return语句中的数据类型
 - 函数体

```
int main(){
// 函数名 main
// 函数参数 void（表示无参数）
// 函数类型 int
// 函数体 {}
}
```

- 常用数据类型
 - 空型（void）：对应函数没有返回值（无return语句）
- 函数的调用
 - 实际参数（调用）→ 形式参数（函数）
- 程序执行顺序
 - 编译过程：从**源文件**第一行开始到**源文件**最后一行结束
 - 函数声明：在main函数前，不包括函数体
 - 函数定义/函数实现：在main函数后，包括函数所有组成部分
 - 运行过程：从**main函数**第一行开始到**main函数**最后一行结束

*1.6 常变量

```
double PI = 3.14;  
const double PI = 3.14; // 常变量PI的数值后续不可修改
```

*1.7 带默认参数的函数

- 无默认参数的函数

```
void func(int a, int b) { cout<<a+b<<endl; }
```

调用时必须给出函数参数a和b的数值

```
func(1,2); // a = 1, b = 2  
func(x,y); // a = x, b = y
```

- 带全部默认参数的函数

```
void func(int a=1, int b=2) { cout<<a+b<<endl; }
```

调用时可选择是否给出函数参数a和b的数值

```
func(); // a = 1, b = 2  
func(3); // a = 3, b = 2  
func(3,4); // a = 3, b = 4
```

- 带部分默认参数的函数

```
void func(int a, int b=2) { cout<<a+b<<endl; }
```

调用时必须给出函数参数a的数值，可选择给出函数参数b的数值

```
func(3); // a = 3, b = 2  
func(3,4); // a = 3, b = 4
```

注意：只有部分参数有默认值时，只能空缺左侧参数的默认值

```
void func(int a, int b, int c=1); // 正确  
void func(int a, int b=2, int c=1); // 正确  
void func(int a, int b=2, int c); // 错误，func(4,5)时a=4, b=5, c没有赋值  
void func(int a=3, int b, int c); // 错误，func(4,5)时a=4, b=5, c没有赋值  
void func(int a=3, int b=2, int c); // 错误，func(4,5)时a=4, b=5, c没有赋值
```

1.8 函数重载

- 在同一作用域内存在多个功能类似的同名函数

```
void add(int a, int b);  
void add(int a, int b, int c);
```

- 使用条件
 - 在同一作用域内

- 函数名相同
- 函数参数不同（数量、数据类型）

*1.9 数组

- 有限个数据类型相同的变量的集合

```
int a[10]; // 10个整型变量的集合，集合名（数组名）为a
float b[5]; // 5个浮点型变量的集合，集合名（数组名）为b
```

*1.10 指针

```
int x;
int *p = &x; // 指针p指向变量x的地址
```

*1.11 new和delete运算符

- 必须使用指针

```
int *p = new int; // 申请一个int的空间，并将指针p指向该空间
delete p; // 清空指针p指向的空间

int *p = new int[n]; // 申请n个int的连续空间，并将指针p指向该空间的首地址
delete[] p; // 清空指针p指向的连续空间
```

*1.12 引用

```
int &ra = a;
float &rb = b;
```

2. 面向对象程序设计思想

使用洗衣机完成洗衣服的过程描述：

- 面向过程思想：
 1. 设计函数（把衣服放进洗衣机），并调用
 2. 设计函数（添加洗衣液），并调用
 3. 设计函数（洗涤），并调用
 4. 设计函数（漂洗），并调用
 5. 设计函数（甩干），并调用
- 面向对象思想：
 1. 确定对象：人、洗衣机
 2. 设定对象属性和功能
 3. 调用人放衣服的功能
 4. 调用人添加洗衣液的功能
 5. 调用洗衣机的洗涤、漂洗、甩干的功能

3. 面向对象程序设计方法

3.1 类和对象

- 对象：一个有具体属性和行为的实体
- 类：对同类对象的概述，描述共同的属性（数据成员：变量）和行为（成员函数：函数）
- 类成员访问权限：
 - public（公开）：类内类外都能修改
 - 类与外界交互的接口
 - private（私有）：类内可以修改，类外只能访问
 - 保护类成员
- 构造函数
 - 与类名相同
 - 无函数类型
 - 功能：
 - 为新创建对象在内存中划分存储空间（无需编辑）
 - 数据成员初始化

```
// 类的示例
class Person
{
private:
    string name;
    int age;
public:
    Person(string s, int n){name = s; age = n;} // 带参构造函数
    Person(){name = "null"; age = 0;} // 无参构造函数
    void set(string s, int n){name = s; age = n;} // 赋值函数
    void show(){cout<<name<<","<<age<<endl;} // 输出函数
};

int main()
{
    Person p1("Liming",18),p2; // 创建Person类对象p1（通过带参构造函数）和p2（通过无参构造函数）
    p1.show(); // 输出为“Liming, 18”
    p2.show(); // 输出为“null, 0”
    p2.set("hanmeimei",17);
    p2.show(); // 输出为“hanmeimei, 17”
    return 0;
}
```

- 类中常用函数
 - 构造函数（带参、无参）
 - 赋值函数
 - 输出函数